

MiniScript Manual

Learn to read, write, and speak
the world's easiest computer language

Joe Strout

MiniScript Maker

Version 2.0

Tuesday, September 29, 2026

Welcome to MiniScript.....	4
Clean, Clear Syntax.....	4
Comments.....	6
Use of Parentheses.....	6
Local and Global Variables	7
Math-Assignment Operators.....	8
MiniScript is Case-Sensitive.....	8
Control Flow	9
Branching with if	9
Looping with for.....	10
Looping with while	11
Break and Continue	11
The Nature of Truth	12
Data Types.....	13
Numbers.....	13
Strings	14
Lists	15
Maps	17
Frozen Lists and Maps.....	18
Type Checking.....	19
Complete Operator List.....	20
Functions and Classes.....	21
Functions.....	21
Nested Functions.....	22
Classes and Objects	24
Extending the Built-In Types	26
Intrinsic Functions	27
Numeric Functions	27
String Functions	28
List Functions	29
Map Functions.....	30
Error Functions	30
System Functions	31
Examples	32
FizzBuzz	32
Filter.....	33

Greatest Common Divisor	33
Maximum Element	34
Titlecase	34
Titlecase (version 2).....	35
Input Validation.....	35

Welcome to MiniScript

*a high-level, object-oriented language
that is easy to read and write*

MiniScript is a modern scripting language designed to be clean, simple, and easy to learn. It was designed from the ground up by borrowing only the best ideas from other languages such as Python, Lua, Basic, and C#. If you know pretty much any other programming language, you'll pick up MiniScript almost immediately.

And if you've never written a line of code in your life, don't panic! MiniScript is the friendliest and most fun way to get started. It's much easier than you probably expect.

Important: MiniScript is designed as an *embedded* programming language. That means you will usually use it inside some other program, such as a video game, or the *command-line MiniScript* program. You should find another document that describes how to access and use MiniScript within that other program. This document only describes the MiniScript language itself, and the intrinsic functions that are common to most MiniScript applications.

Clean, Clear Syntax

Let's jump right in with an example, to see what MiniScript code looks like.

```
s = "Spam"
while s.len < 50
  s = s + ", spam"
end while
print s + " and spam!"
```

Each statement in MiniScript normally occurs on a single line by itself. Notice that there are no semicolons, curly braces, or other markers at the end of a line.

However, there is one exception: if you want to join multiple statements on one line, just to make your code more compact, you can do this by separating the statements with a semicolon. The following code is ugly, but legal.

```
s = "Spam"; while s.len < 50; s = s + ", spam"; end while
print s + " and spam!"
```

In practice this feature is rarely used, but it's there if you need it.

Code Blocks

If you're used to C-derived languages (such as C, C++, C#, etc.), then you're used to seeing curly braces around blocks of code. MiniScript doesn't roll that way; code blocks always begin with a keyword (**if**, **for**, **while**, or **function**) and end with a matching end statement (**end if**, **end for**, **end while**, or **end function**).

Whitespace and Indentation

You can insert spaces and tabs into your code pretty much wherever you want. You can't break up an identifier or keyword (**pr int** is not the same as **print**), nor omit a space between two identifiers or keywords (**end if** is correct, but **endif** would not work). And of course spaces within quotation marks go into your string exactly as you would expect. But between numbers, operators, etc., you can include extra spaces however you like. The following two lines are exactly the same, as far as MiniScript is concerned.

```
x=4*10+2
x = 4 * 10 + 2
```

To make the structure of code more readable, it's traditional to indent the lines within a code block by either a tab or two spaces. But it's not required. MiniScript doesn't care how or whether you indent your code, so do whatever works best for you.

Breaking Long Lines

Unlike C-derived languages, there are no semicolons or other funny punctuation at the end of each line to let the computer know that the statement is over. Instead, the line break alone is enough to signal that. But what if you need to enter a statement longer than one line?

MiniScript will recognize that a statement is incomplete, and continues on the next line, if the last token (before any comment — see below) is an open parenthesis, square bracket, or brace; or a comma; or any binary operator (such as **+**, *****, and so on). So, for example, you could do:

```
speech = ["Four score and seven years ago our fathers",
  "brought forth on this continent, a new nation, conceived",
  "in Liberty, and dedicated to the proposition that all",
  "men are created equal."]
```

That's four lines, but only one statement as far as MiniScript is concerned. That's because the first three lines each ends with a comma, which tells MiniScript that more is coming.

Comments

Comments are little notes you leave for yourself, or other programmers reading your code. They are completely ignored by MiniScript. Comments begin with two slashes, and extend to the end of a line. So you can put a comment either on a line by itself, or after a statement.

```
// How many roads must a man walk down?
x = 6*7 // forty-two
```

Just like indentation, comments are never required... but they're often a good idea!

Use of Parentheses

Parentheses in MiniScript have only three uses:

1. Use them to group math operations in the order you want them, just as in algebra.

```
x = (2+4)*7 // this is different from 2+4*7
```

2. Use them around the arguments in a function call, except when the function call is the entire statement.

```
print cos(0) // parens needed; cannot just say: print cos 0
```

3. Use them when declaring a function that takes parameters (see the Functions chapter).

Since other languages often require parentheses in lots of other places, it's worth pointing out where parentheses are *not* used in MiniScript. First, don't put parentheses around the condition of an **if** or **while** statement (more on these later). Second, parentheses are not needed (and should be omitted) when calling a function without any arguments. For example, there is a **time** function that gets the number of seconds since the program began. It doesn't need any arguments, so you can invoke it without parentheses.

```
x = time
```

Finally, as mentioned above, you don't need parentheses around the arguments of a function that is the very first thing on a statement. The following example prints ten numbers, waiting one second each, and then prints a message. Notice how we're calling **print** and **wait** without any parentheses. But the **range** call, because it has arguments and is used as part of a larger statement, does need them.

```
for i in range(10, 1)
  print i
  wait
end for
print "Boom!"
```

Local and Global Variables

A *variable* is a word (also called an identifier) associated with a value. Think of variables as little boxes that you can store data in. You create a variable simply by assigning a value to it, as in many of the examples we’ve already seen.

```
x = 42
```

This line creates a variable called `x`, if it didn’t exist already, and stores 42 in it. This replaces the previous value of `x`, if any.

Variables in MiniScript are **dynamically typed**; that is, you can assign any type of data (see the chapter on Data Types) to any variable.

Variables are always **local** in scope. That means that a variable called “x” inside one function has nothing at all to do with another variable called “x” in another function; each variable is scoped (restricted) to the current function executing at the time of the assignment.

However, MiniScript also supports code outside of any function, as in all the examples we’ve seen so far. In this context, local and global variables are the same. In other words, assigning 42 to `x` outside of a function creates a global variable called `x`. Such global variables may be accessed from any context.

Note that when a context has a local variable of the same name as a global, an identifier will always resolve to the local variable first. Similarly, a simple assignment statement within a function will always create a local variable, rather than a global one. In cases where you really need to access the global variable instead, there is a `globals` object that provides this access. (See the Intrinsic Functions chapter for more detail on `globals`.)

```
demo = function
  print x          // prints the global x (40)
  x = 2            // creates a local 'x' with a value of 2
  print x          // prints the local x (2)
  print globals.x  // prints the global x again (40)
  globals.x = 42    // reassigns the global x
  print x          // still the local value (2)
  print globals.x  // prints the new global value (42)
end function

x = 40             // creates a global 'x' with a value of 40
demo               // invokes the function above
```

Overuse of global variables can sometimes lead to tricky bugs, so it’s best to use them sparingly and rely on local variables as much as possible. MiniScript is designed so that this good practice is what happens naturally.

Math-Assignment Operators

As a convenient shorthand, the math operators (+, −, *, /, %, and ^) may be used in *math-assignment form*. This does a math operation with a variable, and assigns the result back to that variable. For example, the math-assignment form:

```
x += 1
```

means exactly the same thing as:

```
x = x + 1
```

The only difference is: if the thing you're assigning to requires evaluating an expression, that expression is evaluated only once in math-assignment form, whereas the long form would evaluate it twice. (If that's confusing, don't worry; this distinction rarely matters, and if you're ever in a situation where it does, you will know.)

The previous rules about local and global variables still apply. So, to update a global variable in math-assignment form, you would write something like this.

```
globals.x *= 5
```

This works not only for numbers, but for any data type where the operator used is defined. For example, if the global `x` in the example above were `"ha"`, then after executing that line, the value of `x` would be `"hahahahaha"`.

MiniScript is Case-Sensitive

Uppercase and lowercase matters in MiniScript. The `print` intrinsic function must be typed exactly `print`, and not `Print`, `PRINT`, or any other variation. The same applies to any variables, functions, or classes you define.

While how you use case in your own identifiers is up to you, a common convention is to capitalize classes (e.g. `Shape`), but use lowercase for variables. Thus the following would be a perfectly sensible bit of code.

```
shape = new Shape // create a Shape object called shape
```

While we're on the subject of conventions, in most cases you should avoid starting any global variables or function names with an underscore. Identifiers starting with an underscore are often used by the host environment for special “under the hood” code, and name collisions there could cause problems.

Control Flow

looping and branching

Control flow is how you make code execute multiple times, or execute only under certain conditions. Without it, your scripts would be limited to starting at the first line, executing each line exactly once in order, and ending after the last line.

MiniScript includes one kind of branching (conditional) structure, and two kinds of loops.

Branching with if

Use an **if...then** statement to specify some condition under which the following statements should be executed. The basic syntax is:

```
if condition then
...
end if
```

When the condition is not true, MiniScript will jump directly to the **end if** statement.

```
if x == 42 then
  print "I have found the Ultimate Answer!"
end if
```

The whole set of lines, from **if...then** to **end if**, is known as an *if block*.

Sometimes you want to do something else when the specified condition is not true. You can specify this with an *else block* before the **end if**.

```
if x == 42 then
  print "I have found the Ultimate Answer!"
else
  print "I am still searching."
end if
```

Finally, you can check for additional conditions by adding *else-if blocks* as needed. Here's a slightly more practical example that converts a number to words.

```
if apples == 0 then
  print "You have no apples."
else if apples == 1 then
  print "You have one apple."
else if apples > 10 then
  print "You have a lot of apples!"
else
  print "You have " + apples + " apples."
end if
```

In this case, the first condition that matches will execute its block of lines. If none of the conditions match, then the **else** block will run instead.

Note that for all these forms, the **if**, **else if**, **else**, and **end if** statements must each be on its own line. However, there is also a "short form" **if** statement that allows you to write an **if** or **if/else** on a single line, provided you have only a single statement for the **then** block, and a single statement for the **else** block (if you have an **else** block at all). A short-form **if** looks like this:

```
if x == null then x = 1
```

...while a short-form **if/else** looks like this:

```
if x >= 0 then print "positive" else print "negative"
```

Notice that **end if** is not used with a short-form **if** or **if/else**. Moreover, there is no way to put more than one statement into the **then** or **else** block. If you need more than one statement, then use the standard multi-line form.

Looping with for

A **for...in** statement loops over a block of code zero or more times. The syntax is:

```
for variable in list
...
end for
```

The whole block is referred to as a *for loop*. On each iteration through the loop, the variable is assigned one value from the specified list. You'll learn more about lists in the Data Types chapter, but for now, it's enough to know that you can easily create a list of numbers using the **range** function.

This example counts from 10 down to 1, and then blasts off.

```
for i in range(10, 1)
  print i + "..."
end for
print "Liftoff!"
```

See the **range** function in the Intrinsic Functions chapter for more options on that.

Instead of a list, you can also iterate over a text string. In this case the loop variable will be assigned each character of the string in order.

Finally, it is also possible to iterate over maps. Again, maps will be explained in the Data Types chapter, but just keep in mind that when you use a **for** statement with a map, then on each iteration through the loop, your loop variable is a little mini-map containing **key** and **value**. For example:

```
m = {1:"one", 2:"two", 3:"three"}
for kv in m
  print "Key " + kv.key + " has value " + kv.value
end for
```

This prints out each of the key-value pairs in the map.

Looping with while

The other way to loop over code in MiniScript is with a *while loop*. The syntax is:

```
while condition
  ...
end while
```

This executes the contained code as long as *condition* is true. More specifically, it first evaluates the condition, and if it's not true, it jumps directly to **end while**. If it is true, then it executes the lines within the loop, and then jumps back up to the **while** statement. The process repeats forever, or until the condition becomes false.

This is illustrated by the very first example in this manual, repeated here.

```
s = "Spam"
while s.len < 50
  s = s + ", spam"
end while
print s + " and spam!"
```

This code builds a string (*s*) by adding more spam to it, as long as the string length is less than 50. Once it is no longer less than 50, the loop exits, and the result is printed.

Break and Continue

There are two additional keywords that let you bail out of a while or for loop early. First, the **break** statement jumps directly out of the loop, to the next line past the **end for** or **end while**. Consider the following.

```
while true      // loops forever!
  if time > 100 then break
end while
```

Whenever you see **while true** (or **while 1**, which is equivalent), it is an infinite loop — *unless* there is a **break** statement in the body of the loop. As soon as that **break** statement executes, we jump directly out of the loop. It works for **for** loops in exactly the same way. In the case of nested loops, **break** breaks out of only the innermost loop.

The **continue** statement skips the rest of the body of the loop, and proceeds with the next iteration. This is often used for "bail-out" cases in a large loop, where under certain conditions you want to skip an iteration and just go on with the next one.

```
for i in range(1,100)
  if i == 42 then continue
  print "Considering " + i + "..."
end for
```

This will print out the numbers 1 through 100, *except* for 42, which is skipped. Note that if you simply changed **continue** to **break** in this example, the loop would print the numbers 1 through 41, and then stop.

The Nature of Truth

We have talked about evaluating conditions as true or false, without explaining what that really means. Usually you don't need to worry about it, but here are the details anyway.

Boolean (true/false) values in MiniScript are represented as numbers. When conditions are evaluated for **if** and **while** statements, a value of 0 (zero) is considered false; any other value is considered true. In fact the built-in keywords **true** and **false** are exactly equivalent to the numbers 1 and 0 respectively.

When you use comparison operators such as **==** (equal), **!=** (not equal), **>** (greater than), and **<=** (less than or equal), these compare their operands and evaluate to either 1 (if true) or 0 (if false).

See the Numbers section of the Data Types chapter for more boolean operations you can apply to numbers (including **and**, **or**, and **not**).

Finally, in a context that demands a truth value — that is, in an **if** and **while** statement, or as an operand of **and**, **or**, and **not** — other data types will be considered false if they are empty, and true if they are not empty. So an empty string, list, or map is equivalent to 0 (zero), and any non-empty string, list or map is equivalent to 1 in these contexts. The special value **null** is always considered false.

Data Types

things you can store and manipulate

Variables in MiniScript are dynamically typed; you can store any type of data in any variable. But what types of data are there? In MiniScript, there are four primary data types: *numbers*, *strings*, *lists*, and *maps*. There are a couple of other more obscure types, such as function and null. Everything else, including classes and objects, is actually a special case of a map.

Numbers

All numeric values in MiniScript are stored in standard full-precision format (also known as “doubles” in C-derived languages). Numbers are also used to represent true (1) and false (0).

Numeric literals are written as ordinary numbers, e.g. **42**, **3.1415**, or **-0.24**.

You can use the following operators on numbers (where *a* and *b* are numbers).

<code>a + b</code>	addition	numeric sum of a and b
<code>a - b</code>	subtraction	numeric difference of a and b
<code>a * b</code>	multiplication	a multiplied by b
<code>a / b</code>	division	a divided by b
<code>a % b</code>	modulo	remainder after dividing a by b
<code>a ^ b</code>	power	a raised to the power of b
<code>a and b</code>	logical and	<code>a * b</code> , clamped to the range [0,1]
<code>a or b</code>	logical or	<code>a + b - a*b</code> , clamped to the range [0,1]
<code>not a</code>	negation	<code>1 - abs(a)</code> , clamped to the range [0,1]
<code>a == b</code>	equality	1 if a equals b, else 0
<code>a != b</code>	inequality	1 if a is not equal to b, else 0
<code>a > b</code>	greater than	1 if a is greater than b, else 0
<code>a >= b</code>	greater than or equal	1 if a is greater than or equal to b, else 0
<code>a < b</code>	less than	1 if a is less than b, else 0
<code>a <= b</code>	less than or equal	1 if a is less than or equal to b, else 0

Note that **and**, **or**, and **not** are not functions; they are operators, and go between (or in the case of **not**, before) their operands just like all the other operators.

You can check whether a variable contains a number with the **isa** operator. There is an intrinsic class called **number**, and **x isa number** returns **true** (1) whenever *x* is, in fact, a number.

Strings

Text values in MiniScript are stored as strings of Unicode characters. String literals in the code are enclosed by double quotes (**"**). Be sure to use ordinary straight quotes, not the fancy curly quotes some word processors insist on making.

If your string literal needs to include quotation marks, you can do this by typing the quotation marks twice. For example:

```
s = "If you do not help us, we shall say ""Ni"" to you."
```

Strings may be concatenated with the **+** operator, and if you try to add a number and a string together, the number will be automatically converted to a string and then concatenated. Strings may also be replicated (repeated) or cut down to a fraction of their former selves, by multiplying or dividing them by a number.

```
s = "Spam" * 5    // SpamSpamSpamSpamSpam
s = s / 2         // SpamSpamSp
```

The full set of string operators is shown below; *s* and *t* are strings, and *n* and *m* are numbers.

s + t	concatenation	string formed by concatenating <i>t</i> to <i>s</i>
s - t	subtraction (chop)	if <i>s</i> ends in <i>t</i> , returns <i>s</i> with <i>t</i> removed; otherwise just returns <i>s</i>
s * n	replication	<i>s</i> repeated <i>n</i> times (including some fractional amount of <i>s</i>)
s / n	division	equivalent to <i>s</i> * (1/ <i>n</i>)
s[n]	index	character <i>n</i> of <i>s</i> (<i>all indexes are 0-based; negative indexes count from end</i>)
s[:n]	left slice	substring of <i>s</i> up to but not including character <i>n</i>
s[n:]	right slice	substring of <i>s</i> from character <i>n</i> to the end
s[n:m]	slice	substring of <i>s</i> from character <i>n</i> up to but not including character <i>m</i>
s == t	equality	1 if <i>s</i> equals <i>t</i> , else 0 (<i>all string comparisons are case-sensitive</i>)
s != t	inequality	1 if <i>s</i> is not equal to <i>t</i> , else 0
s > t	greater than	1 if <i>s</i> is greater than (collates after) <i>t</i> , else 0
s >= t	greater than or equal	1 if <i>s</i> is greater than or equal to <i>t</i> , else 0
s < t	less than	1 if <i>s</i> is less than (collates before) <i>t</i> , else 0
s <= t	less than or equal	1 if <i>s</i> is less than or equal to <i>t</i> , else 0

The table above does not include **and**, **or**, and **not**, but these operators work perfectly well on strings through boolean coercion (see "The Nature of Truth" in the previous chapter). In any boolean context, `s` is considered true if it contains any characters, and false if it is the empty string.

Also not listed is behavior of the **isa** operator with strings. There is an intrinsic type called **string**, and `s isa string` returns **true** (1) for any string `s`.

The slice operators deserve a bit of explanation. The basic syntax is `s[n:m]`, which gets a substring of `s` starting at character `n`, and going up to (but not including) character `m`, where we number characters starting from 0. But this basic syntax is extended with a handful of neat tricks:

1. You may specify just a single index, leaving out the colon, to get a single character. Thus `s[0]` is the first character, `s[1]` is the second, etc.
2. You may use a negative index, and it will count from the end. So `s[-1]` is the last character, `s[-2]` is the next-to-last, etc. This works for any of the slice indexes.
3. You may omit the first index from the two-index form, and it will default to 0. This is a handy way to get the first `n` characters of a string. So `s[:3]` returns the first 3 characters of `s`; `s[:-3]` returns all but the last three characters of `s`.
4. You may omit the last index from the two-index form, and it will continue to the end of the string. Thus, `s[3:]` skips the first three characters and returns the rest of the string.

The way these indexes work results in a lot of very handy properties. For example, `s[:n] + s[n:] == s` for any value of `n` from 0 through `s.len`; in other words, there is a very natural syntax for splitting a string into two parts, which is a fairly common thing to do.

Finally, note that strings are **immutable**; just like numbers, you can never change a string, but you can create a *new* string and assign it to an existing variable. The following example shows one correct and one incorrect way to change "spin" into "spun".

```
s = "spin"
s = s[:2] + "u" + s[3:]    // OK
s[3] = "u"                // no can do (Runtime Error)
```

Lists

The third basic data type in MiniScript is the *list*. This is an ordered collection of elements, accessible by index starting with zero. Each element of a list may be any type, including another list.

You define a list by using square brackets around the elements, which should be separated with commas.

```
x = [2, 4, 6, 8]
```

The code above creates a list with four elements and assigns it to `x`. But again, list elements don't have to be numbers; they can also be strings, lists, or maps. Here's another example.

```
x = [2, "four", [1, 2, 3], {8:"eight"}]
```

Working with a list is very much like working with a string. You can concatenate two lists with `+`, replicate or cut a list with `*` and `/`, and access elements or sublists using the same slice syntax. Here are the operators valid on lists, where `p` and `q` are lists, and `n` and `m` are numbers.

<code>p + q</code>	concatenation	list formed by concatenating <code>q</code> to <code>p</code>
<code>p * n</code>	replication	<code>p</code> repeated <code>n</code> times (including some fractional amount of <code>p</code>)
<code>p / n</code>	division	equivalent to <code>p * (1/n)</code>
<code>p[n]</code>	index	element <code>n</code> of <code>p</code> (<i>all indexes are 0-based; negative indexes count from end</i>)
<code>p[:n]</code>	left slice	sublist of <code>p</code> up to but not including element <code>n</code>
<code>p[n:]</code>	right slice	sublist of <code>p</code> from element <code>n</code> to the end
<code>p[n:m]</code>	slice	sublist of <code>p</code> from element <code>n</code> up to but not including element <code>m</code>

In addition, you can use `x isa list` to check whether any variable `x` contains a list.

The slice operators work exactly the same way as with strings. So `p[-1]` is the last element of list `p`; `p[3:]` skips the first three elements and returns the rest of the list, and so forth.

However, there is one important difference: lists are **mutable**. You can change the contents of a list (by assigning to `p[n]` or using one of the list methods like `p.push`), and no matter how many different variables are referring to that list, they will all see the change. The following example illustrates.

```
a = [1, 2, 3]    // creates a list and assigns to a
b = a           // assigns that SAME list to b
a[-1] = 5       // changes the last element of our list to 5
print b         // prints: [1, 2, 5]
```

Because `a` and `b` both refer to the same list, any changes (*mutations*) made to that list can be seen from either variable.

If you want to be sure you have a fresh copy of a list, rather than a shared reference, a common trick is to use `[:]` to make a slice that includes the entire list. This copies the elements into a new list. Compare the following example to the previous one.


```
a = [1, 2, 3]    // creates a list and assigns to a
b = a[:]        // assigns a COPY of that list to b
a[-1] = 5       // changes the last element of our first list to 5
print b         // prints: [1, 2, 3] (our copy hasn't changed)
```

Maps

The final basic data type in MiniScript is the *map*. A map is a set of key-value pairs, where each unique key maps to some value. In some programming environments, this same concept is called a *dictionary*.

Create a map with curly braces around a comma-separated list of key-value pairs. Specify each pair by separating the key and value with a colon, as shown here.

```
m = {1:"one", 2:"two", 3:"three"}
```

The map created here contains three key-value pairs, each mapping a number to a string (which happens to be the English word for that number in this example).

Map keys can be any data type, and must be unique; if you reuse a key, the previous value is replaced. Values may be any type, and don't need to be unique. Maps preserve the order in which key/value pairs were added; **for** loops iterate over a map this same order.

Maps support only a handful of operators (*d* and *e* are maps, *k* is a key, and *v* is a value):

<code>d + e</code>	concatenation	map formed by assigning <code>d[k] = v</code> for every <code>k,v</code> pair in <code>e</code>
<code>d[k]</code>	index	value associated with key <code>k</code> in <code>d</code>
<code>d.k</code>	dot index	value associated with (string) <code>k</code> in <code>d</code>

There are two ways to get and set members of a map. The first is to use the square-brackets index operator, just as with strings or lists, except that in the case of a map, the key can be any data type, not just a number.

```
d = {"yes":"hai", "no":"iie", "maybe":"tabun"}
print d["maybe"]    // prints: tabun
d["maybe"] = "kamo"
print d["maybe"]    // prints: kamo
```

The second way is using the *dot indexer*. This works only in the special case where the key is a string that is a valid identifier: it begins with a letter, and contains only letters, numbers, and underscores. In this case you can write the key after a dot (period) rather than enclosing it in square brackets and quotation marks — the key essentially becomes an identifier in the language. The following is functionally equivalent to the previous example.

```
d = {"yes":"hai", "no":"iie", "maybe":"tabun"}
print d.maybe        // prints: tabun
d.maybe = "kamo"
```

```
print d["maybe"]      // prints: kamo
```

This dot indexer is mostly syntactic sugar that makes accessing elements of a map easier to read and write. But there are some subtle differences in cases where the map represents a class or object, as described in the next chapter.

Finally, like the other basic types, there is an intrinsic class that represents maps — **map** in this case. So `x isa map` will return true for any map (including any class or object, as you'll see in the next section).

Frozen Lists and Maps

We said above that lists and maps are *mutable*, that is, their contents can be changed. This is usually true, but not always: a list or map may be *frozen*, which makes it immutable. You can manually freeze a list or map with the **freeze** operator:

```
m = {1:"one", 2:"two"}
freeze m
m[1] = "uno"  // Runtime error: Attempt to modify a frozen map
```

Alternatively, you can ask for a frozen copy of a list or map using the **frozenCopy** function. Note that this will *not* make a copy if given something that's already frozen; it will just return the same immutable list or map in this case.

```
m = {1:"one", 2:"two"}
mCopy = frozenCopy(m)
m[1] = "uno"      // A-OK
mCopy[1] = "uno"  // Runtime error: Attempt to modify a frozen map
```

If you use a (normal, unfrozen) list or map as a key in another map, it will actually store a frozen copy of your list or map as the key. (This is necessary because mutating a value used as a map key would cause problems with retrieval later; storing a frozen copy ensures that no such mutation of a map key can happen.) You can avoid this extra copy by freezing the key yourself before storing it in the map.

If you ever need to check whether a list or map is frozen, you can do so with the **isFrozen** function:

```
x = [1, 2, 3]
print isFrozen(x)  // prints 0, since x is not yet frozen
freeze x
print isFrozen(x)  // prints 1, since x is frozen
```

Type Checking

The `isa` operator was mentioned several times above. This is how you can check, at runtime, what sort of data you have. In many cases you won't care, thanks to MiniScript's automatic type conversion. But sometimes you do.

Suppose for example you want to make a method that prints its argument surrounded by parentheses... but if the caller passes in a list, then you want to join the elements of that list with commas. You could accomplish that with `isa`.

```
spew = function(x)
  if x isa list then x = x.join(", ")
  print "(" + x + ")"
end function

spew 42                // prints: {42}
spew [18, 42, "hike!"] // prints: {18, 42, hike!}
```

You can also find out more details about any value by using the `info` function, which returns a map of information about the value. This map always includes “type” and may include other details, depending on what type it is. Here are some examples:

```
print info(@range)
{"type": "funcRef", "name": "range", "note": "", "sourceLoc":
  null, "params": [{"name": "from", "default": 0}, {"name":
    "to", "default": 0}, {"name": "step", "default": null}],
  "closure": 0}
print info(range(1,10))
{"type": "list", "computed": 1, "frozen": 0}
```

Note that you will often want to use `@` with `info` if your variable might refer to a function, assuming you want to learn about the function itself and not whatever value it returns.

Complete Operator List

The table below shows all the operators in the MiniScript language, along with their precedence. Operands in an expression chain will always be grouped by higher-precedence operators before lower-precedence ones; e.g., $x + y * z$ is processed as $x + (y * z)$, because the $*$ operator is higher precedence than the $+$ operator.

Operator	Meaning	Precedence
$A = B$	assignment	0
$A \text{ or } B$	logical OR: true if either operand is true	1
$A \text{ and } B$	logical AND: true if both operands are true	2
not A	logical negation: true if its operand is false, and vice versa	3
$A \text{ isa } B$	type checking	4
$A == B$	equality comparison: true if operands are equal	5
$A != B$	inequality comparison: true if operands are not equal	5
$A > B$	greater-than comparison	5
$A < B$	less-than comparison	5
$A >= B$	greater-than or equal-to comparison	5
$A <= B$	less-than or equal-to comparison	5
$A + B$	addition or concatenation	6
$A - B$	subtraction or string trimming	6
$A * B$	multiplication or replication	7
A / B	division or reduction	7
$A \% B$	modulo (remainder)	7
-A	unary minus (numeric negation)	8
new A	instantiation	9
@A	address-of (reference function without invoking it)	10
$A ^ B$	power: A raised to the power of B	11
A[B]	indexing	12
A[B:C]	slicing	12
A(B, C...)	calling a function	12
A.B	dot operator	12

Functions and Classes

the building blocks of sophisticated software

A *function* is essentially a sub-program that does some particular task. We've already seen some of the functions built into MiniScript, such as `time` and `range`, and even `print`. There are many more of those, which will be documented in the next chapter. But the real power of a programming language comes from defining your own functions.

Beyond that, as a program grows in size and complexity, it becomes useful to start organizing it into *classes*. A class is basically a collection of functions and data, where *objects* of a class share the same functions but may have unique data.

Functions

A function in MiniScript is a special data type, at the same level as numbers, strings, lists, and maps. You can define a function with the `function` keyword, assign it to a variable, and then invoke it via that variable, just like the built-in functions. Here's an example.

```
triple = function(n=1)
  return n*3
end function
print triple      // prints: 3
print triple(5)   // prints: 15
```

This declares a function that triples any value given to it, and assigns that function to a variable called `triple`. The triple function is then invoked, with and without an argument.

The syntax for declaring a function is:

```
function(parameters)
...
end function
```

where *parameters* is a comma-separated list of zero or more parameters, each of the form *name* or *name=defaultValue*. When a function is invoked, arguments will be matched up to the functions by position. If fewer arguments are given than parameters are defined, the remaining parameters are given their default values — and if no default value was defined for that parameter, then it is set to `null`.

Note that the parentheses after the `function` keyword are required only if there are parameters. In the case of a function with no parameters, the parentheses are not required (and by standard convention, should be omitted).

It's important to understand that a function is itself a bit of data. It's just that, whenever looking up the value of a variable, MiniScript checks for this special function data type; and if found, it invokes that function, rather than returning the function itself.

Usually that is exactly what is wanted, as in the example above. But occasionally you may want to copy the function reference, rather than invoking the function. You can do this by prepending your identifier with an `@` (read “address of”). Example:

```
triple = function(n=1)
  return n*3
end function
x = @triple
print x(5)      // prints: 15
```

Here we've again declared a function and stored it in a variable called `triple`. Then we copy the *address of* that function into another variable called `x`. At this point we can invoke the function either way, via `triple` or via `x`, and both do exactly the same thing. Had we left out the `@` on the assignment, MiniScript would have instead evaluated the function `triple` refers to, and assigned the result (3) to `x`.

Here's a more realistic example. We'll define a function called `apply` which can apply a given function to each element of a list. Then we can invoke this on a list with any function, simply by using `@` to refer to the function we want to apply.

```
apply = function(lst, func)
  result = lst[:]           // make a copy of the list
  for i in indexes(result)
    result[i] = func(result[i]) // apply func to each element
  end for
  return result             // return modified result
end function

print apply([1, 2, 3], @triple) // prints: [3, 6, 9]
```

To summarize, you invoke a function by simply using any identifier that refers to it. You avoid this invocation, and refer instead to the function itself, by putting `@` before the identifier.

Nested Functions

MiniScript allows you to define functions within functions. This is an advanced feature that most users may never need, but it can come in handy on occasion, especially in conjunction with something like the “apply” method above. Just as with any other local value, you might want to avoid cluttering the global namespace just for a function that you only use in one place. Here's a simple example that assumes we have the `apply` method defined above.

```
doubleAll = function(lst)
  f = function(x)
    return x + x
  end function
  return apply(lst, @f)
end function
```

So inside the function referred to by the (global variable) **doubleAll**, we define another function, and assign it to the (local variable) **f**. Then we pass that function in as the second argument to the **apply** function (or more pedantically, to the function referred to by the **apply** global variable).

When you have a nested function like this, it can access the local variables of the function that contains it. Just as with global variables, it can do this without any prefix (as long as there isn't some local variable with the same name getting in the way). But to assign to a variable of the outer function, you must use the special identifier **outer**. Here's an example.

```
makeList = function(sep)
  counter = 0
  makeItem = function(item)
    outer.counter = counter + 1
    return counter + sep + item
  end function
  return [makeItem("a"), makeItem("b"), makeItem("c")]
end function

print makeList(". ")
```

Here, **makeList** refers to the outer function, and **makeItem** is the inner (nested) function. Notice how **makeList** has a local variable called **counter**, initialized to 0. But the inner function both reads that value, and updates it using **outer.counter**. Work through this code carefully to see if you can figure out what it prints... and then try it and see if you were right!

Again, this nested-function business is an advanced feature which beginners can safely forget about. But for advanced users, it is a language feature worth understanding.

There are two features of functions that exist solely to help you remember what they are about. First, if a function is defined in an assignment statement (as is usually the case), the left-hand side of that statement is stored on the function as its name. Second, if the first line of the function body is a constant string (including a string expression that can be worked out at compile time), that is stored on the function as its note. The name and note can be retrieved via the **info** function (see page 19).

Classes and Objects

MiniScript supports object-oriented programming (*OOP*) via prototype-based inheritance. That is, there is fundamentally no difference in MiniScript between a class and an object; the difference, when it exists at all, lives solely in the intent of the programmer.

A class or object is a map with a special `__isa` entry that points to the parent (prototype). This is an implementation detail you rarely need to worry about, because it is handled automatically by the following rules:

1. When you create a map using the special `new` operator, the `__isa` member is set for you.
2. When you look up an identifier in a map, MiniScript will walk the `__isa` chain looking for a map containing that identifier. The value returned is the first value found.
3. Finally, the `isa` operator also walks the `__isa` chain, and returns true if any map in that chain matches the right-hand operand. In other words, `x isa y` returns `true` if `x` is any subclass of `y`.

These simple rules provide almost everything needed for object-oriented programming. A series of "classes" may be defined as maps containing functions and default data, which are inherited or overridden as needed. An "object" is just another map, inherited from some class, which normally contains only custom data.

Let's illustrate with an example. We'll define a class called `Shape`, with a subclass called `Square`.

```
Shape = {}
Shape.sides = 0

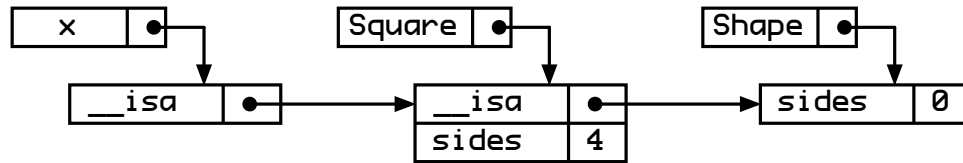
Square = new Shape
Square.sides = 4
```

A base class is just an ordinary map; in this case, we added a `sides` entry with a value of 0, signifying that "sides" is a bit of data we expect every `Shape` to have. Then we created a subclass by using `new Shape`, and assigned this to `Square`. In `Square`, we overrode the value of `sides` (as all squares should have 4 sides).

Now let's create an instance of our `Square` class, again by using `new`.

```
x = new Square
print x.sides      // prints: 4
```

Notice how we're using the traditional OOP terminology of "class" and "instance" for convenience, but in reality, there are just three maps — `Shape` is the prototype of `Square`, and `Square` is the prototype of `x`. The `__isa` member of each map points to the prototype, because we created them with `new`.



Now let's add a function to the Shape class, which should work for any shape subclass or object.

```

Shape.degrees = function
  return 180 * (self.sides - 2)
end function

print x.degrees    // prints: 360

```

This example illustrates one additional rule important to object-oriented programming:

- When a function is invoked via dot indexing, it receives a special **self** variable that refers to the object on which it was invoked.

So in the example above, we invoked the **degrees** function as **x.degrees**, which looks for a member called “degrees” in **x** (and its prototypes via the **__isa** chain). And when that function is invoked, a special local variable called **self** is bound to the **x** object, i.e. the first map in the search chain. This allows class functions to access object data.

There is just one more bit of special support for object-oriented programming, and that is the **super** keyword. This is another built-in variable (similar to **self**) defined when you invoke a method via dot syntax, but when you call another method via **super**, it invokes that method on the base class, while keeping **self** bound to the same value as in the current function. In other words, **super** lets you call a superclass method, even if you've overridden it. Continuing the previous example, suppose we want to define a subclass of Square that always has 42 more degrees than nonmagical shapes would have:

```

MagicSquare = new Square
MagicSquare.degrees = function
  return super.degrees + 42
end function

y = new MagicSquare
print y.degrees    // prints: 402

```

Notice how the **MagicSquare.degrees** function calls **super.degrees**. That causes MiniScript to walk the **__isa** chain, looking for the first implementation of **degrees** it can find. That would be **Shape.degrees**, so it invokes that, with a **self** still bound to **y**.

Extending the Built-In Types

There are maps that represent each of the basic data types: `number`, `string`, `list`, and `map`. These contain the built-in methods for those types. By adding new methods to one of these maps, you can add new methods usable with dot syntax on values of that type.

For example, while there are built-in string methods `.upper` and `.lower` to convert a string to upper- or lower-case, there isn't a method to capitalize a string — that is, convert only the first letter to uppercase. But you could add such a method in your program as follows.

```
string.capitalized = function
  if self.len < 2 then return self.upper
  return self[0].upper + self[1:]
end function
```

The function itself is fairly simple: if our string (`self`) is less than 2 characters long, just uppercase the whole thing; otherwise uppercase the first letter, and append the rest. But because we have assigned this function to `string.capitalized`, that is, added it to the `string` map, we can call it with dot syntax on any string.

```
print "miniScript".capitalized    // prints: MiniScript
```

Some common MiniScript environments, including *Mini Micro*, *command-line MiniScript*, and *raylib-miniscript*, have libraries of functions that extend the built-in types, giving them substantially more functionality. Look for `stringUtil`, `listUtil`, `mapUtil`, or the equivalents in whatever environment you are using.

Intrinsic Functions

built-in functions you can rely on

MiniScript comes with a standard set of built-in (or *intrinsic*) functions. Many of these are accessed like globals (though they actually live in a special map called `intrinsic`). Others (particularly functions intended for use with strings, lists, and maps) are normally invoked via dot syntax after an identifier.

In fact, though, all intrinsic functions that use dot syntax are written in such a way that they can *also* be invoked as global functions. So, for example, you can get the length of a string `s` by typing `s.len`, but you can also do the same thing as `len(s)`.

The following tables list the standard intrinsic functions, divided by data type on which they operate. Keep in mind that MiniScript is intended to be embedded in some host environment, such as a game or application. The host will normally add additional intrinsic functions particular to that environment. Please consult the documentation or help materials for your host environment for information on these extra functions.

Numeric Functions

MiniScript includes a selection of trigonometric functions, which all work in radians (rather than degrees), and other math functions, as well as random numbers and conversion of numbers into strings.

In the following table, *x* is any number, *i* is an integer, and *r* is a number of radians.

<code>abs(x)</code>	absolute value of <i>x</i>
<code>acos(x)</code>	arccosine of <i>x</i> , in radians
<code>asin(x)</code>	arcsine of <i>x</i> , in radians
<code>atan(y, x=1)</code>	arctangent of <i>y/x</i> , in radians (returns correct quadrant if optional <i>x</i> parameter is used)
<code>bitAnd(x, y)</code>	treats <i>x</i> and <i>y</i> as integers, and returns bitwise "and" of <i>a</i> and <i>b</i>
<code>bitOr(x, y)</code>	treats <i>x</i> and <i>y</i> as integers, and returns bitwise "or" of <i>a</i> and <i>b</i>
<code>bitXor(x, y)</code>	treats <i>x</i> and <i>y</i> as integers, and returns bitwise "exclusive or" of <i>a</i> and <i>b</i>
<code>ceil(x)</code>	next whole number equal to or greater than <i>x</i>
<code>char(i)</code>	returns Unicode character with code point <i>i</i> (see string <code>.code</code> for the inverse function)
<code>cos(r)</code>	cosine of <i>r</i> radians
<code>floor(x)</code>	next whole number less than or equal to <i>x</i>

<code>log(x, base=10)</code>	logarithm (with the given base) of x, i.e., the value y such that $\text{base}^y == x$
<code>pi</code>	3.14159265358979
<code>range(x, y=0, step=null)</code>	returns a list containing values from x through y, in increments of step; step == null is treated as a step of 1 if y > x, or -1 otherwise
<code>round(x, d=0)</code>	x rounded to d decimal places
<code>rnd(seed=null)</code>	if seed=null, returns random number in the range [0,1); if seed != null, seeds the random number generator with the given integer value
<code>sign(x)</code>	sign of x: -1 if x < 0; 0 if x == 0; 1 if x > 0
<code>sin(r)</code>	sine of r radians
<code>sqrt(x)</code>	square root of x
<code>str(x)</code>	converts x to a string
<code>tan(r)</code>	tangent of r radians

String Functions

All string functions except `slice` are designed to be invoked on strings using dot syntax, but can also be invoked as globals with the string passed in as the first parameter. Note that strings are immutable; all string functions return a *new* string, leaving the original string unchanged. In the following table, *self* refers to the string, *s* is another string argument, and *i* is an integer number.

<code>.code</code>	Unicode code point of first character of self (see numeric <i>char</i> function for inverse)
<code>.hasIndex(i)</code>	1 if i is in the range 0 to self.len-1; otherwise 0
<code>.indexes</code>	<code>range(0, self.len-1)</code>
<code>.indexOf(s, after=null)</code>	0-based position of first substring s within self, or null if not found; optionally begins the search after the given position
<code>.insert(index, s)</code>	returns new string with s inserted at position 0
<code>.len</code>	length (number of characters) of self
<code>.lower</code>	lowercase version of self
<code>.remove(s)</code>	self, but with first occurrence of substring s removed (if any)
<code>.replace(oldval, newval, maxCount=null)</code>	returns a new string with up to maxCount occurrences of substring oldval replaced with newval (if maxCount unspecified, then replaces all occurrences)
<code>.upper</code>	uppercase version of self
<code>.val</code>	converts self to a number (if self is not a valid number, returns 0)
<code>.values</code>	list of individual characters in self (e.g. "spam".values = ["s", "p", "a", "m"])

<code>slice(s, from, to)</code>	equivalent to <code>s[from:to]</code>
<code>.split(delimiter=" ", maxCount=null)</code>	splits the string into a list by the given delimiter, with at most <code>maxCount</code> entries (if <code>maxCount</code> is unspecified, then splits into a list of any size)

List Functions

All list functions except **`slice`** are designed to be invoked on lists using dot syntax, but can also be invoked as globals with the list passed in as the first parameter. Lists are mutable; the **`pop`**, **`pull`**, **`push`**, **`shuffle`**, and **`remove`** functions modify the list in place. To use a list like a stack, add items with **`push`** and remove them with **`pop`**. To use a list like a queue, add items with **`push`** and remove them with **`pull`**.

In the following table, *self* is a list, *i* is an integer, and *x* is any value.

<code>.hasIndex(i)</code>	1 if <i>i</i> is in the range 0 to <code>self.len-1</code> ; otherwise 0
<code>.indexes</code>	<code>range(0, self.len-1)</code>
<code>.indexOf(x, after=null)</code>	0-based position of first element matching <i>x</i> in <i>self</i> , or null if not found; optionally begins the search after the given position
<code>.insert(index, value)</code>	inserts <i>value</i> into <i>self</i> at the given index (in place)
<code>.join(delimiter=" ")</code>	builds a string by joining elements by the given delimiter
<code>.len</code>	length (number of elements) of <i>self</i>
<code>.pop</code>	removes and returns the last element of <i>self</i> (like a stack)
<code>.pull</code>	removes and returns the first element of <i>self</i> (like a queue)
<code>.push(x)</code>	appends the given value to the end of <i>self</i> ; often used with <code>pop</code> or <code>pull</code>
<code>.shuffle</code>	randomly rearranges the elements of <i>self</i> (in place)
<code>.sort(key=null, ascending=1)</code>	sorts list in place, optionally by value of the given key (e.g. in a list of maps)
<code>.sum</code>	total of all numeric elements of <i>self</i>
<code>.remove(i)</code>	removes element at index <i>i</i> from <i>self</i> (in place)
<code>.replace(oldval, newval, maxCount=null)</code>	replaces (in place) up to <code>maxCount</code> occurrences of <i>oldval</i> in the list with <i>newval</i> (if <code>maxCount</code> not specified, then all occurrences are replaced)
<code>slice(list, from, to)</code>	equivalent to <code>list[from:to]</code>

Map Functions

Functions on maps are very similar to functions on lists. Maps (like lists) are mutable; the **push**, **pop**, **remove**, and **shuffle** methods modify the map in place. You can treat a map like a set using **push**, which inserts 1 (true) for the value of the given key, and **pop**, which returns a key and removes it (and its value) from the map. Keep in mind that the order of keys in a map is undefined.

In the following table, *self* is a map, *i* is an integer, and *x* is any value.

<code>.hasIndex(x)</code>	1 if <i>x</i> is a key contained in <i>self</i> ; 0 otherwise
<code>.indexes</code>	list containing all keys of <i>self</i> , in arbitrary order
<code>.indexOf(x, after=null)</code>	first key in <i>self</i> that maps to <i>x</i> , or null if none; optionally begins the search after the given key
<code>.len</code>	length (number of key-value pairs) of <i>self</i>
<code>.pop</code>	remove and return an arbitrary key from <i>self</i>
<code>.push(x)</code>	equivalent to <code>self[x] = 1</code>
<code>.remove(x)</code>	removes the key-value pair where <code>key=x</code> from <i>self</i> (in place)
<code>.replace(oldval, newval, maxCount=null)</code>	replaces (in place) up to <code>maxCount</code> occurrences of value <code>oldval</code> in the map with <code>newval</code> (if <code>maxCount</code> not specified, then all occurrences are replaced)
<code>.shuffle</code>	randomly remaps values for keys
<code>.sum</code>	total of all numeric values in <i>self</i>
<code>.values</code>	list containing all values of <i>self</i> , in arbitrary order

Error Functions

When writing error-handling code, remember that **error** is the name of the error type, while **err** is the function you use to create a new error value. When you create a new error value, you can optionally wrap an “inner” error — use this when you’ve caught some generic error (for example, a file error) and want to wrap it in a more specific context (*e.g.*, your high-scores module). You can also call **err** on an existing error value to make a subtype or instance (a new error that “is a” instance of the base type).

<code>err(msg, inner=null)</code>	make a new error value with the given message (and optional inner error)
<code>e.err(msg, inner=null)</code>	make a new error subtype/instance of base type <i>e</i>

System Functions

The following functions relate to the operation of MiniScript itself, or interact with the host environment. The latter (**print**, **input**, **time**, and **wait**) are only quasi-standard, in that support for them depends on the host application, and so they may not function in some environments.

<code>globals</code>	reference to the global variable map
<code>intrinsic</code>	a map containing all the global intrinsic functions
<code>locals</code>	reference to the local variable map for the current call frame
<code>print(x, delim)</code>	convert <code>x</code> to a string and print to some text output stream, optionally followed by <code>delim</code> ; if <code>delim</code> is not specified, the output is followed by a line break in most environments
<code>input(prompt)</code>	print the prompt, then wait for the user to type a line of input and press enter/return; return whatever they entered (as a string)
<code>refEquals(a,b)</code>	returns 1 if <code>a</code> and <code>b</code> refer to the same instance (not just equal values)
<code>stackTrace</code>	returns the current call stack, as a list of strings
<code>time</code>	number of seconds since program execution began
<code>wait(x=1)</code>	wait <code>x</code> seconds before proceeding with the next MiniScript instruction
<code>yield</code>	wait for next invocation of main engine loop (e.g., next frame in a game)
<code>freeze x</code>	make <code>x</code> (a list or map) frozen (immutable), if it's not already
<code>frozenCopy(x)</code>	return <code>x</code> if it is frozen, or if not, return a copy of <code>x</code> that is frozen
<code>isFrozen(x)</code>	returns 1 if <code>x</code> is a frozen list or map, 0 otherwise
<code>info(x)</code>	return a map of details about value <code>x</code>
<code>gc</code>	reference to the garbage collection module (useful methods: <code>.collect</code> and <code>.stats</code>)

Examples

small programs that do interesting things

While we’ve given short examples of MiniScript code throughout this manual, this chapter presents several longer, more interesting examples. Many of the tasks illustrated are taken from RosettaCode, an online database of programming challenges with solutions in many languages. You can go there to compare the MiniScript solution to any other language; you may be amazed how much more readable MiniScript is than the alternatives.

FizzBuzz

FizzBuzz is a standard introductory-level programming challenge¹. The task is simple: print the numbers 1 through 100, *but*: for multiples of three, print “Fizz” instead of the number; for multiples of five, print “Buzz” instead of the number, and for any number that’s a multiple of three *and* five, print “FizzBuzz”.

There are clearly many ways to tackle this; here’s one.

```
1. fizzBuzz = function(n)
2.   for i in range(1, n)
3.     s = "Fizz" * (i%3==0) + "Buzz" * (i%5==0)
4.     if s == "" then s = str(i)
5.     print s
6.   end for
7. end function
8. fizzBuzz 100
```

Instead of just hard-coding a loop from 1 to 100, we’ve made a function that can FizzBuzz up to any number. Within that function, the only clever bit is line 3, which takes advantage of a couple of MiniScript features. First, comparisons (such as `i%3==0` — read “i mod 3 equals zero”) evaluate to 1 when true, or 0 when false. Second, you can multiply a string by a number to repeat it that many times. This means that if you multiply a string by a condition, you get either the original string (if the condition is true) or the empty string (if it is false).

That lets us easily generate “Fizz”, “Buzz”, and “FizzBuzz” depending on what our loop counter is divisible by. Line 4 simply fills in the number if we don’t get one of those strings. (Quiz: can you rewrite this line to use the same multiply-by-condition trick as line 3?)

¹ <http://rosettacode.org/wiki/FizzBuzz>

Filter

Here's another RosettaCode task²: select certain elements from an Array into a new Array in a generic way. To demonstrate, select all even numbers from an Array.

```

1. filter = function(seq, f) // filter seq to where f is true
2.   result = []
3.   for i in seq
4.     if f(i) then result = result + [i]
5.   end for
6.   return result
7. end function
8.
9. isEven = function(x)
10.  return x % 2 == 0
11. end function
12.
13. list = [2,3,5,6,8,9]
14. print filter(list, @isEven)

```

This is a pretty straightforward conversion of the task description into MiniScript code. Our **filter** function takes a list and a function, and builds a new list by appending each element where the function, applied to that element, is true.

We illustrate by making an **isEven** function that returns true only when its argument mod 2 is zero (i.e., the argument is evenly divisible by 2). Then we pass **@isEven** to find just the even elements of a given list.

Greatest Common Divisor

Here's a function that finds the biggest number that can divide evenly into two given numbers³. Middle schoolers everywhere will soon be out of work.

```

1. gcd = function(a, b)
2.   if a == 0 then return b
3.   while b != 0
4.     newA = b
5.     b = a % b
6.     a = newA
7.   end while
8.   return abs(a)
9. end function
10. print gcd(-21, 35)

```

The algorithm here, known as the “Euclidian algorithm for finding the GCD,” is clever. The actual MiniScript code is simple.

² <http://rosettacode.org/wiki/Filter>

³ http://rosettacode.org/wiki/Greatest_common_divisor

Maximum Element

MiniScript does not have a standard intrinsic for finding the maximum element of a list. But you can easily add it yourself, using this code.

```
1. max = function(seq)
2.   if seq.len == 0 then return null
3.   max = seq[0]
4.   for item in seq
5.     if item > max then max = item
6.   end for
7.   return max
8. end function
9. print max([5, -2, 12, 7, 0])
```

Pretty simple stuff. Line 2 checks to make sure the sneaky user hasn't given us an empty list; if they have, we return null, as there is no sensible max in that case. Otherwise, we just suppose it's the first element, and then loop over each element in the list, keeping the biggest.

Notice that the `max` variable assigned to on line 1 is in the global variable space, while the `max` assigned on lines 3 and 5 (and then returned on line 7) is local to a function. These happen to have the same name, but have nothing to do with each other. As a matter of style, it might have been better to name the local variable `result` rather than `max`. But it seemed like a good opportunity to demonstrate how local and global variables are separate, even if they have the same name.

Titlecase

MiniScript has intrinsics to convert a string to all upper- or lower-case letters. But what if you want to capitalize just the first letter of each word, and lowercase the rest?

```
1. titlecase = function(s)
2.   result = ""
3.   for i in s.indexes
4.     if i == 0 or s[i-1] == " " then
5.       result = result + s[i].upper
6.     else
7.       result = result + s[i].lower
8.     end if
9.   end for
10.  return result
11. end function
12. print titlecase("SO LONG and thanks for all the fish")
```

We just iterate over the string, capitalizing each letter that is either the very first character in the string, or is preceded by a space, and lower-casing the rest.

Titlecase (version 2)

The previous version of Titlecase works fine, but is somewhat suboptimal, because it grows a string by adding to it character by character. This recopies the earlier characters in the string many times. The following code shows a better way.

```
1. titlecase = function(s)
2.   result = s.split("")
3.   for i in s.indexes
4.     if i == 0 or s[i-1] == " " then
5.       result[i] = s[i].upper
6.     else
7.       result[i] = s[i].lower
8.     end if
9.   end for
10.  return result.join("")
11.end function
12.print titlecase("SO LONG and thanks for all the fish")
```

Here we start by splitting the string into characters (by using the empty string as the delimiter to split on). Then we iterate over the string, updating each character in our list, and join them back together at the end.

Input Validation

The **round** intrinsic will automatically convert a string to a number, but if the given string is not a valid number, it returns an **error**. This example shows how to catch that and keep trying, in an environment that supports **input**.

```
13.while true
14.  age = round(input("Enter your age: "))
15.  if age isa error then print age.message else break
16.end while
17.print "That's " + age*7 + " in dog years!"
```